EDIAGD

EVERY DAY IS A GREAT DAY

INTERIM MOU · SECTION 4

October 2026 Report

EDIAGD Contribution Record

Pear Tree Companies, LLC

Period covered October 1 to October 31, 2026

Hours recorded 125.4

Value at $175 per hour $21,945

Out-of-pocket $173.30

Prepared November 4, 2026 · Ryan Kelly

SECTION 4 RECORD

Summary

Pear Tree contributed an estimated **125.4 hours** of development and operations to the Venture in October 2026, worth **$21,945** at the $175 hourly rate quoted in the July 5, 2026 proposal. Out-of-pocket costs were **$173.30**.

Kept under Section 4 of the Interim MOU dated October 1, 2026, which requires each party to record its Venture contributions, including development hours at a rate to be agreed.

OCTOBER 2026

Period	October 1 to October 31, 2026
Active working days	22
Commits	74 distinct across all branches
Production releases	43 pull requests merged to main
App Store	EDIAGD 1.0.0 submitted for review, October 26
Hours	125.4
Rate	$175 per hour (July 5, 2026 proposal)
Value	$21,945

Source is the git history of the ediagd repository, read across all branches rather than main alone, plus file timestamps for work done but not yet committed. The git history doubles as the deployment record: every production release on Vercel is a merge to main, so the 43 pull request merges are the 43 production deploys. Vercel's own records would add build timestamps a few minutes after each merge and the preview deploys raised per pull request, neither of which changes the hours.

Nothing here has been invoiced. This is a record of contribution under MOU Section 4, to be credited, reimbursed, or otherwise addressed in the definitive agreements.

WHAT IS MEASURED AND WHAT IS ESTIMATED

Of the 125.4 hours, 81.8 are measured from commit spans and 43.6 are estimated from the scope of the day's work or from file timestamps. Every commit in this repository is reviewed and approved by hand, so commits tend to land together after a working session rather than spreading through it. That makes the commit span a floor on every day rather than a measure of it. The two hours for the App Store submission rest on recollection, because that work happens in Apple's console and leaves no trace in the repository. The Mux invoice covering October was issued on October 29 and paid on October 30, so the cost figures are settled rather than estimated.

What was built

Eleven workstreams. All shipped to production during the month.

1. Video trim pipeline (Oct 5 to Oct 14), the largest single effort

Built an instrument to measure dead air at the head and tail of every published film, then cut it. Measured all 447 films and found 254 carrying 65 minutes of dead air. Cut 252 in the main pass, then 34 more at the greeting, then re-cut 3 from local masters. Added `trim:reconcile` after three manual recoveries, because a refusal is not a record. Built verification that reads the rendition a phone actually plays, using energy measurement for how much air and Whisper transcription for which word, requiring both passes to agree before ruling. Caught and fixed four faults that raised no error, a derive step that read its own output before writing, and two films whose vertical cut did not match the landscape.

2. Lesson library and film playback (Oct 3 to Oct 5)

The library plays its films; the film moved out of the card deck and above it. One watched rule, so credit comes from `met` and no page prints a percentage. Migration 0160 so the library shows what it has. Full library audit, report only, nothing applied. Openers, closers and the Walk-Around films placed.

3. Rollcall (Oct 10, 13, 19)

The six facts requested, including a column that had never been written. Posts once, at the end of the day. Posts only on days the store is open.

4. Push notifications and re-engagement (Oct 1, 12, 15)

Two trigger reasons: protect a streak, or bring someone back. Measured whether the push actually brings them back by joining three records, which surfaced one that never fires. Corrected the three-minute copy to five minutes across every screen and the push text.

5. Manager features (Oct 13, 14, 20, 26, 29)

Manager Meetings route, grant, and the entitlement check that never said yes. Manager Bank Account: 17 films ingested, trimmed and published, plus a verifier that had been blind to them. Manager roster drops advisors who have left. Team screen shows whether each advisor trained today and which videos they completed, counted as watched rather than accrued. Volume 4 closed the month: a MANAGER ingest route with an op code that is not allowed to touch content, and migrations 0168 and 0169 so manager films carry their own placement rather than inheriting it.

6. Front door (Oct 3 to Oct 4)

Signed-out public preview of a morning, using the brand lockup. Settled at two films after trying four. Film stills generated to a public bucket with permanent URLs.

7. Onboarding and account creation (Oct 1)

First sign-in creates the `app_user` row. Mitch's welcome film opens screen one. Onboarding shows a sentence rather than a constraint. Feedback screen wired to Slack and email.

8. Provisioning (Oct 12)

`provision:roster` provisions a whole store, then invites.

9. Supporting infrastructure (throughout)

Transcript backfill reading captions from the playback id the app serves. A read-only window on the corpus for the site's post engine. `db:migrate` refuses to run unless on an up-to-date main. Certs page progress bar per track. Launch runbook step 1 repaired. On Oct 30, day completion moved onto evidence: a day is saved by the evidence the request carries, not by a call that may never land.

10. App Store submission (Oct 26 to Oct 28)

Prepared EDIAGD 1.0.0 for App Store review. Repaired the Apple review account on Oct 26, which would have failed review: it had no `work_schedule` row, so every screen redirected to onboarding, and its email was never confirmed. Both are the common first rejection. Gave it a seven-day schedule deliberately, since a weekday-only schedule would have shown App Review a rest card on a weekend and been read as a broken screen. Wrote a sign-in verifier that proves the gate as an authenticated user rather than as the service role, after three prior incidents where a gate keyed on `auth.uid()` shut the backend out of its own product. Generated screenshot sets at three device sizes, 19 images. Wrote the App Store Connect runbook covering app information, pricing, privacy declarations, age rating and the version page, then submitted EDIAGD 1.0.0 for review on October 26.

11. Audio sync, diagnosis and repair (Oct 28 to Oct 30) REWORK

Advisors reported that films cut by the October trim pass play with audio and picture out of sync. Confirmed against an untrimmed control: trimmed films drift from -228 ms to +1,240 ms with no constant offset, so the cut itself was misaligning the streams, picture snapping to a keyframe while audio cut at the exact sample. Built a sync scan across all 462 films and wrote the repair brief on Oct 28. Repaired it on Oct 30: one judge for sync and a gate so the fault cannot recur, the vertical now cut at the landscape's window so the gate sees the same picture, supporting instruments for master alignment and vertical proof, two vertical derives at a time with every temp file tagged to its slot, and the job stopped by a file rather than a signal. Originals survived every cut, so nothing was lost. The fault came from the trim pass in workstream 1, so this is rework rather than new feature work, and it is recorded at 14.8 hours like any other work in the month.

Day by day

Session span is the time between the first and last commit of the day. Hours are the figure carried into the total; where they differ from the span, the next section explains why. Days marked **clustered** had their commits approved together in one sitting, so the span measures the approval and not the day, and the hours are estimated from the scope of what shipped. Days marked **files** produced no commits, and the hours come from file timestamps.

DATE	COMMITS	SESSION SPAN	HOURS	PRINCIPAL WORK
Oct 1	7	6.8h	6.8	Push triggers, onboarding, feedback screen, five-minute copy
Oct 2	2	2.1h	2.1	Corpus window for the site, transcript playback bridge
Oct 3	5	3.2h	3.2	Library plays films, front door, film stills
Oct 4	10	4.6h	4.6	Front door iterations, library audit, migrate guard
Oct 5	8	12.1h	12.1	Trim measure pass, watched rule, migration 0160
Oct 6	3	7.7h	7.7	Trim reconcile, transcript force pass
Oct 7	4	8.2h	8.2	252 films cut, verification instrument
Oct 8	4	4.1h	4.1	447 verified on phone rendition, derive guard
Oct 9	4	10.7h	10.7	Final trim batches, Certs progress bar
Oct 10	1	clustered	6.0	Rollcall, 31 files
Oct 11	1	clustered	2.0	CAF-002 reshoots, kiosk film split
Oct 12	4	clustered	4.0	Provision roster
Oct 13	2	clustered	3.0	Manager Meetings entitlement, Rollcall timing
Oct 14	2	6.3h	6.3	Manager Bank Account run, mindset trio re-cut
Oct 15	1	clustered	6.0	Push effectiveness measurement, 14 files
Oct 19	1	clustered	4.0	Rollcall on open days, 10 files
Oct 20	2	clustered	3.0	Roster departures, ruling records
Oct 26	5	7.2h	9.2	Team screen, review account repair, launch runbook, **App Store submission (2.0, stated)**

DATE	COMMITS	SESSION SPAN	HOURS	PRINCIPAL WORK
Oct 27	none	files	3.5	Screenshot sets at two further device sizes, Rollcall post brief
Oct 28	none	files	6.0	Audio sync diagnosis, scan across 462 films, repair brief
Oct 29	2	0.9h	4.1	MANAGER ingest route, migrations 0168 and 0169, manager film placement
Oct 30	6	8.8h	8.8	Sync judge and gate, vertical cut at the landscape window, day-completion evidence
Oct 31	none	files	0	One instrument run only, no hours recorded
Total	**74**		**125.4**	

No activity on Oct 16, 17, 18, or 21 through 25.

How the hours were calculated

No timer was running in October, so these are reconstructed from the commit record and file timestamps rather than logged. The method is below so that anyone can check it. Of the 125.4 hours, 81.8 are measured from commit spans and 43.6 are estimated.

Every commit here is approved by hand, which is why spans are a floor. Ryan reviews and approves each commit rather than letting them land automatically, so a day's commits often arrive together at the end of a working session instead of spreading through it. The span between the first and last commit therefore measures when the approvals happened, not how long the day was. On twelve days the approvals are spread widely enough that the span is informative and it is used as it stands. On the rest it collapses, and the day is estimated from what shipped. Nothing is added on top of a span to cover the work that preceded it.

Days read from their span. Twelve days have commits spread across hours, and the span between the first and last is taken as the working session. October 30 shows how conservative this is: its span runs 14:06 to 22:54, but file timestamps show work from 11:08, almost three hours earlier, and that time is not counted.

Long days are recorded in full. Oct 5 runs 12.1 hours and Oct 9 runs 10.7. An earlier draft capped both at 9 on the assumption that a span that long must contain meals and other work. That assumption was wrong: EDIAGD days sometimes run from waking until midnight, so the cap was deleting real hours. It has been dropped and both days stand at their measured span, which adds 4.8 hours against the earlier draft.

Days estimated from scope. Seven days have their commits clustered into minutes for the reason above, so there is no usable span. These are estimated from what the day produced, at 2 to 6 hours each, totaling 28 hours. Oct 10 alone touched 31 files and added 4,268 lines.

Days with no commits, read from file timestamps. Oct 27 and 28 produced no commits. The hours come from timestamps on the files written those days: three screenshot sets, two briefs, a sync scan still running, and uncommitted edits to the Today screen, Manager screen and the Rollcall cron route.

Oct 29 is read from file timestamps rather than its span. Its two commits were approved 53 minutes apart at the end of the day, so the 0.9 hour span describes the approvals and not the day, which produced an ingest route, a Mux webhook change and two migrations. File timestamps show hand-authored work from 17:15 to 23:30, with an idle gap of 2.1 hours between 18:10 and 20:18. The 4.1 hours recorded is that window less the gap.

Oct 31 is recorded at zero, and that is a judgement. There are no commits. The only evidence is 28 temporary measurement files written between 08:09 and 08:14, the output of one run of the sync instrument. No source file was touched and nothing was committed. A batch job writes those files whether or not anyone is at the keyboard, so five minutes of machine output is not a working day. If Ryan recalls working that morning, the hours should be added.

The App Store submission is Ryan's own figure. Filling in the listing, privacy declarations, age rating and version page, setting the review account password and uploading the build all happen in Apple's console and Xcode, and leave no trace in this repository. Ryan records it as 2 hours on October 26, the day EDIAGD 1.0.0 went in for review. It is the one line here that rests on recollection rather than evidence, which is the ordinary case for everything that happens outside a repository. If any of that work happened inside the 16:14 to 23:29 commit window, the day is overstated by up to 2 hours.

Two method faults found and corrected during the month. The first pass walked only the main branch and so missed commits on branches that were never merged, including the October 11 reshoot work. A later pass read file timestamps in UTC, which put October 30 evening work on October 31 and would have invented an active day that did not exist. All times in this report are US Central, matching the commit record. The lesson in both cases is that a commit log is not a complete record of effort, which is the argument for logging hours directly.

Value

BASIS	CALCULATION	AMOUNT
Hourly, as recorded	125.4 hours × $175	**$21,945**
Retainer plus overage	$3,500 + (117.4 hours × $175)	$24,045

Both figures come from the July 5, 2026 proposal, quoted at arm's length months before this structure was discussed. The hourly basis is the lower of the two and is the one recorded.

The retainer line is shown only as a cross-check. Under the proposal, the Pilot and Launch tier is $3,500 per month and includes 8 enhancement hours, with additional work at $175. October's work was almost entirely feature development rather than maintenance, so the straight hourly basis is the cleaner characterization and the one to carry into the definitive agreements.

MOU SECTION 4

Out-of-pocket costs

MOU Section 4 requires approved out-of-pocket costs alongside hours. These were carried by Pear Tree in October. Figures are from the EDIAGD Expenses workbook in Drive. Only October falls under the MOU; August and September predate the October 1 effective date and belong to the contributed build, the same as the September iOS shell work.

The table reflects the settled basis: infrastructure that runs the product is a Venture cost, and development tooling is not.

ITEM	OCT 2026	BASIS
Supabase	$9.68	Invoice line item, org total $48.26
Vercel	$10.66	$85.28 over 16 projects times 2, two charges in the month
Mux	$152.96	Invoice 10/29, usage Sep 29 to Oct 29: encoding $133.20, storage $30.29, less $20.00 credits, plus $9.47 tax. Paid Oct 30
Venture total	**$173.30**	

THE MUX BILLING PERIOD DOES NOT MATCH THE MONTH

The October invoice covers usage from September 29 to October 29, so it carries the last two days of September and misses the last two days of October. The sync re-cuts on October 30 and 31 re-encoded films after the meter closed and will appear on the November invoice. Mux is the one line where the venture cost and the calendar month will always be a couple of days out of step.

Development tooling is not a Venture cost. Model usage and other tooling spent writing the software is Pear Tree's cost of production, absorbed by Pear Tree and already covered by the $175 rate. It does not appear in this record.

The product itself runs no AI. What an advisor is shown each morning is computed from their own attach rates against the store average, weighted by revenue. No model sits in that path, which the repository confirms: it carries no Anthropic SDK dependency, no API key variable and no model references. So there is no Claude API pass-through to expect going forward, and the Venture's recurring infrastructure is Supabase, Vercel and Mux. That is a cleaner cost base than the July proposal assumed when it scoped Ask Eddie as a Claude-powered assistant with its own usage.

Model usage that generated content shipping inside the product, such as coaching cues or quiz drafts, would be a Venture content contribution rather than an expense. None was generated in October, so there is nothing to record under that heading.

FOR THE RECORD

Notes and caveats

Code volume is not the measure, and is not claimed as one. October added roughly 89,000 lines and removed 23,000. Of the additions, about 52,700 are JSON: generated measurement reports, transcripts and trim ledgers, which are machine output, not authorship. Hand-written source is closer to 28,000 lines across TypeScript, SQL and shell, plus 8,200 lines of documentation and handoff notes. Hours are the measure used here. Line counts appear only as supporting detail.

This is AI-assisted development. The work was done with Claude Code under Pear Tree's direction. The hours are real hours of specification, review, debugging and running pipelines, but nobody should read 28,000 lines of source as 28,000 lines typed by hand. Stating this plainly is better than having it raised later.

Mitch's contributions are not recorded here. Section 4 obliges both parties. His content development, film production and subject-matter work over the same period belong in a parallel record that he keeps. This document covers Pear Tree only and should not be read as a complete account of what the Venture received in October.

Scope. Only commits to the ediagd repository are counted. Time spent on calls, Mitch's content review, DMS data questions, and commercial work such as the partnership structure is not included, and under the proposal's terms most of that is not billable development anyway.

The iOS shell predates the MOU. The native wrapper, push entitlement, launch animation, app icon and TestFlight builds 1 through 12 were built between September 7 and September 23, before the MOU's October 1 effective date. That work belongs to the contributed platform build, not to this record. It is worth noting because the App Store submission in October rests on it, and because it is part of what the roughly $200,000 contribution bought.

Reconciliation against the repository's own work log

The repository carries its own record of October in `docs/WORK_LOG.md`, a reconstructed block of 16 days totaling 94 hours covering October 2 to 26. That figure and this report's 125.4 look like a contradiction. They are not.

BASIS	HOURS	NOTE
Work log, the 16 days it covers	94.0	A supplied monthly total, apportioned across days by commit volume
This report, those same 16 days	94.2	Each day derived independently from spans and timestamps
Difference on shared days	0.2	Two independent methods landing in the same place
Days the work log never covered	31.2	Oct 1, 11, 27, 28, 29, 30, 31
This report, full month	**125.4**	

The whole gap is seven days the work log never had. Day by day the two differ by as much as 5.4 hours in both directions and the differences cancel, which is what should be expected: the work log states in its own header that its 94 was a supplied total spread across days by commit volume, and that no per-day timing evidence sits behind any number in its hours column. The agreement at the month total is meaningful; the per-day agreement is not. The work log's October block is superseded by this report rather than contradicted by it, and should be retired.

One thing in the work log is simply wrong. It records that git showed nothing at all on October 1 and asks whether that day was real. October 1 carries 7 commits on main, with author and committer dates matching, including the push trigger work and the first-sign-in account creation. The day was never missing from the repository; it was dropped by the read. That is the same class of fault as the branch-only walk and the UTC timestamp error already noted above, and it is the third instance of it.

The commit count, settled

October carries **74 distinct commits** across all branches, grouped by author date. A raw count returns 76, because two October 12 commits were cherry-picked and appear on more than one branch. **43 pull requests** were merged to main, and each one is a production deploy on Vercel. A 44th merge commit sits in main's history from October 26, but it merged origin/main into a feature branch rather than releasing anything, so it is not a deploy. None of this touches the hours, which come from spans and timestamps rather than from counting commits.

Why the work log is not being used

It was never wired up. `scripts/work-log.mjs`, `scripts/work-log-report.mjs` and `docs/WORK_LOG.md` are all untracked and not ignored either, so they exist on one Mac and in no clone of this repository. `package.json` registers no `log` or `log:report` script, so the `npm run log` that the file documents in its own header does not exist and fails the first time anyone runs it. The tool was written on October 29 and never connected. Committing those three files and adding two lines to `package.json` is the whole fix, and it is the one change that would stop every future month costing a reconstruction like this one.

Going forward. Reconstruction worked for one month and will get harder as the record lengthens. From November, log hours as the work happens, even roughly. A dated line per session is enough and removes the argument entirely.

EDIAGD is operated by Pear Tree Companies. Films by Mitch Hardt. This report is kept under Section 4 of the Interim MOU dated October 1, 2026 and reflects the repository and expense records as read on November 4, 2026.